



## 3.2. Sorting Data Values

[Previous Section:](#)

 [3.1. Data Filtering](#)

[Contents:](#)

[1. Basic Sorting](#)

[1.1. Sorting by Column Values](#)

[1.2. Sorting by Index](#)

[2. Duplicate Values in Sorting](#)

[Summary](#)

[References](#)

When analyzing data, the order of rows often matters. For example, you may want to identify the students with the highest GPAs, list records alphabetically by name, or organize data by age. This process is known as **sorting**.

Pandas provides convenient methods for sorting a DataFrame by either its **data values** or its **index**. Sorting does not change the actual data—it simply rearranges the order in which the rows or columns are displayed.

In this section, we will learn how to:

- Sort a DataFrame by one or more columns.
- Control whether the sorting is in ascending or descending order.
- Sort a DataFrame by its index.
- Understand how duplicate values affect the sorting order.

Sorting is one of the most common data manipulation operations and is frequently used before filtering, grouping, or visualizing data.

We continue using the synthetic dataset below.

```

import numpy as np
import pandas as pd

# Reproducible randomness
np.random.seed(42)

names = [
    "James", "Chris", "Adam", "Marian", "Peter",
    "Kevin", "Linda", "Emma", "Sophia", "Daniel",
    "Noah", "Olivia", "Lucas", "Grace", "Henry",
    "Liam", "Ava", "Mia", "Nathan", "Chloe"
]

majors = ["AI", "IT", "CSC", "SW"]

n = 20

df = pd.DataFrame({
    "Name": np.array(names),
    "Age": np.random.randint(18, 21, n),
    "Major": np.random.choice(majors, n),
    "GPA": np.round(np.random.uniform(2.0, 4.0, n), 2)
})

print(df.head(10))

```

## 1. Basic Sorting

Pandas provides two primary methods for sorting a DataFrame.

| Method                        | Description                                       |
|-------------------------------|---------------------------------------------------|
| <code>df.sort_values()</code> | Sorts the DataFrame by one or more column values. |
| <code>df.sort_index()</code>  | Sorts the DataFrame by its row or column index.   |

By default, both methods sort in **ascending order**.

### 1.1. Sorting by Column Values

The most common method is `df.sort_values(by="column_name")`

- **Example 1: Sort by GPA**

```
df.sort_values(by="GPA")
```

Students are arranged from the lowest GPA to the highest.

- **Example 2: Sort in Descending Order**

To reverse the order, set `ascending=False`.

```
df.sort_values(by="GPA", ascending=False)
```

This is useful when ranking students from highest GPA to lowest.

- **Example 3: Sort by Multiple Columns**

Sometimes multiple rows share the same value in one column. We can specify additional columns to break ties.

```
df.sort_values(by=["Major", "GPA"])
```

The DataFrame is first sorted by **Major**. Within each major, students are then sorted by **GPA**.

Each column can also have its own sorting direction.

```
df.sort_values(by=["Major", "GPA"], ascending=[True, False])
```

This sorts majors alphabetically while displaying the highest GPA first within each major.

## 1.2. Sorting by Index

Instead of sorting by data values, we can sort by the DataFrame's index.

```
df.sort_index()
```

This becomes especially useful after setting a meaningful column as the index. For example,

```
df_index = df.set_index("Name")
# Then,
df_index.sort_index()
```

This sorts the DataFrame alphabetically by student name because the **Name** column is now the index.

**Note:** `sort_index()` is most useful after customizing the index. With the default integer index ( `0, 1, 2, ...` ), sorting by index usually has little visible effect.

## 2. Duplicate Values in Sorting

It is common for multiple rows to have the same value in the sorting column.

- For example, `df.sort_values(by="GPA")` simply sorts the students by their GPA. However several students may have the same GPA. When duplicate values occur, Pandas groups those rows together. However, **their relative order is not guaranteed unless additional sorting columns are specified**.
- Therefore, `df.sort_values(by=["GPA", "Name"])` first sorts students by **GPA**. If two students have the same GPA, they are then sorted alphabetically by **Name**.
- You are not limited to two columns. Pandas allows you to sort by **any number of columns**, applying them from left to right as tie-breakers.

```
df.sort_values(by=["Major", "Age", "GPA", "Name"])
```

In this example, Pandas:

1. Sorts by **Major**.
2. If two students have the same major, sorts by **Age**.
3. If both the major and age are the same, sorts by **GPA**.
4. Finally, if all three values are identical, sorts alphabetically by **Name**.

In principle, you can even sort by **every column** in the DataFrame to produce a completely deterministic ordering.

Using multiple sorting columns is recommended whenever you need consistent and reproducible results, especially when generating reports or comparing

datasets.

## Summary

This section introduces to arranging values in data with sorting techniques.

- `sort_values()` sorts a DataFrame by one or more column values.
- By default, sorting is performed in ascending order.
- Setting `ascending=False` sorts values in descending order.
- Multiple columns can be used to create hierarchical sorting and break ties.
- `sort_index()` sorts the DataFrame according to its index and is most useful after assigning a meaningful index.
- When duplicate values exist in the sorting column, adding additional sorting columns produces a consistent and predictable ordering.

***To practice further, complete Lab 03 of the Data Analytics Hands-On Lab.***

 [Lab 07: Data Analytics with Python](#)

## References

[https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.sort\\_values.html](https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.sort_values.html)

[https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.sort\\_index.html](https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.sort_index.html)

[https://www.geeksforgeeks.org/pandas/python-pandas-dataframe-sort\\_values-set-1/](https://www.geeksforgeeks.org/pandas/python-pandas-dataframe-sort_values-set-1/)

[https://www.geeksforgeeks.org/python/python-pandas-dataframe-sort\\_index/](https://www.geeksforgeeks.org/python/python-pandas-dataframe-sort_index/)

## Next Section:

 [3.3. Replacing & Mapping Values](#)